

Finite Automata (FA) Program Documentation

Overview

The Finite Automata (FA) program is designed to read and process a finite automaton described in a text file. It can determine if the automaton is deterministic and check if a given input sequence is accepted by the automaton.

Class Description

FA Class

The FA class encapsulates the representation and operations of a finite automaton. It provides methods to load an automaton from a file, print its details, check if it is deterministic, and verify if a sequence is accepted.

Attributes:

`states`: A list of all states in the automaton.

`alphabet`: A list of all symbols in the automaton's alphabet.

`transitions`: A list of transitions, where each transition is represented as a tuple - is a struct equivalent to (`from`, `symbol`, `to`).

`initialState`: The initial state of the automaton.

`finalStates`: A list of final states of the automaton.

Methods:

`FA(std::string filename)`: Constructor that initializes the FA by loading data from a specified file.

`void loadFromFile(std::string filename)`: Loads the automaton details from a given file.

`void print()`: Prints the automaton's states, alphabet, transitions, initial state, and final states.

`bool isDeterministic()`: Checks if the automaton is deterministic.

`bool isAccepted(std::string sequence)`: Checks if a given sequence is accepted by the automaton.

Input File Format

The automaton is described in a text file with specific keywords defining its components. Below is an example of such a file (FA.in):

```
states = {A, B, C}
epsilon = {0, 1}
initial = A
final = {C}
delta = {(A, 0, A), (A, 0, B), (A, 1, B), (B, 1, C), (C, 1, C)}
```

BNF Representation

The Backus-Naur Form (BNF) for the input file format is as follows:

```
<file> ::= <states> <epsilon> <initial> <final> <delta>

<states> ::= "states" "=" "{" <state-list> "}"
<state-list> ::= <state> | <state> "," <state-list>
<state> ::= <identifier>

<epsilon> ::= "epsilon" "=" "{" <symbol-list> "}"
<symbol-list> ::= <symbol> | <symbol> "," <symbol-list>
<symbol> ::= <character>

<initial> ::= "initial" "=" <state>

<final> ::= "final" "=" "{" <state-list> "}"

<delta> ::= "delta" "=" "{" <transition-list> "}"
<transition-list> ::= <transition> | <transition> "," <transition-list>
<transition> ::= "(" <state> "," <symbol> "," <state> ")"
```

EBNF Representation

The Extended Backus-Naur Form (EBNF) for the input file format is as follows:

```
file = states, epsilon, initial, final, delta;

states = "states", "=", "{", state-list, "}";
state-list = state, {"", state};
state = identifier;

epsilon = "epsilon", "=", "{", symbol-list, "}";
symbol-list = symbol, {"", symbol};
symbol = character;

initial = "initial", "=", state;

final = "final", "=", "{", state-list, "}";

delta = "delta", "=", "{", transition-list, "}";
transition-list = transition, {"", transition};
transition = "(" , state, ",", symbol, ",", state, ")";
```

Integration in Scanner:

The previous version of `Scanner.cpp`, the one found in lab3, used the methods `isConstant` and `isIdentifier` when processing tokens (inside the method `processToken`), which had the corresponding logic defined in our language for identifying constants and identifiers respectively. That has now been replaced with `faIntegerConstant.isAccepted(word)` and `faIdentifier.isAccepted(word)` respectively, which are two methods from FAs which reside inside the `Scanner` class. They are initialized inside the constructor of the `Scanner` as follows:

```
faIdentifier = FA("FA_identifier.in");  
  
faIntegerConstant = FA("FA_integerConstant.in");
```

The `Scanner` class has also suffered other changes - `isConstant` has now been replaced by `isStringConstant` and `isIntegerConstant`; the former uses the same logic as in lab3.

The contents of these two files are the following:

```
FA_identifier.in:  
states = {q0, q1, qf}  
epsilon = {a, b, c, d, e, f, g, h, i, ... as defined in the language  
initial = q0  
final = {qf}  
delta = {(q0, _, q1), (q0, A, qf), (q0, B, qf), (q0, C, qf), (q0, D, qf), (q0,  
E, qf), (q0, F, qf), (q0, G, qf), (q0, H, qf), (q0, I, qf), ... as defined in  
the language
```

```
FA_integerConstant:  
states = {q0, zero, sign, number}  
epsilon = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, +, -}  
initial = q0  
final = {zero, number}  
delta = {(q0, zero, 0), (q0, sign, +), (q0, sign, -), (q0, number, 1), (q0,  
number, 2), (q0, number, 3), (q0, number, 4), ... as defined in the language
```